Lab Work Assignment 1: Implementing a basic machine learning training pipeline

Course: ML Engineering

Objective:

The goal of this lab is to design, implement, and evaluate a supervised learning model using a deep learning approach. Students are expected to work with a large dataset, create a comprehensive training pipeline, and follow best practices in code management and development.

Supplementary resources:

- Sample of full pipeline with code blocks to be updated - https://colab.research.google.com/drive/1KeZq80U4JMFAvbABpgm5Rkt97a5RaIlb?usp=sharing
- Students will need to update the example with the following:
    - Select their own dataset to download using download_and_extract()
    - Update load_labels() to work with their labels format and convert it to dataframe of format

image_path	label
/content/data/images_dir/image_1.jpg	1

    - Update SimpleNN() to better fit the selected problem (SimpleNN is a very small network that won't achieve good metrics)
    - **Add additional metrics such as accuracy, precision, recall, F1 score in test_model()**

Part 1: Preparing the Dataset

1. **Data Selection and Preprocessing:**
    - **Task:** Choose a large dataset relevant to a supervised learning problem. The dataset should be split into 2-3 parts (train/validation/test) without significantly degrading model performance.
    - **Data Download:** Implement a stage in the pipeline for downloading or accessing the dataset.
    - **Data Ingestion:** Write code to ingest the data and perform an initial split into training, validation, and test sets.
2. **Data Augmentation/Feature Engineering:**
    - **Task:** Apply data augmentation techniques (for image data) or feature engineering (for other data types) to improve model performance.
    - **Optional:** Experiment with different augmentation techniques and document their impact on model performance.

Part 2: Developing the Training Pipeline

1. **Pipeline Structure:**
    - **Task:** Implement a training pipeline with the following stages:
        - **Data Ingestion and Splitting:** Ensure that the data is properly loaded and split.
        - **Training Loop:** Develop a loop to train the model over multiple epochs, including loss calculation and backpropagation.
        - **Evaluation:** Incorporate an evaluation step to assess model performance on validation and test sets after each epoch.
        - **Artifact Collection:** Save the best-performing model, log metrics, and collect any other relevant artifacts (e.g., training logs, visualizations).
2. **Metrics and Evaluation:**
    - **Task:** Define and calculate multiple performance metrics appropriate for your task. Examples include accuracy, precision, recall, F1-score, or others specific to the problem domain.

Part 3: Best Practices and Code Management

1. **Configuration Management:**
    - **Task:** Define configuration files to control various stages of the pipeline, such as data paths, hyperparameters, and model settings. Ensure that changes can be made easily by adjusting configuration files rather than hard-coding values.
2. **Logging:**
    - **Task:** Use Python's `logging` module to track important events in your code, such as the start and end of training, model evaluations, and errors. Avoid using `print()` statements for these purposes.
3. **Code Quality:**
    - **Task:** Implement code quality checks using linters and formatters like `mypy`, `ruff`, `black`, and `isort`. Ensure that your code adheres to good practices, including proper type hints for function and method definitions.
4. **Dependency Management:**
    - **Task:** Use `poetry` to manage project dependencies, ensuring that your project environment is reproducible and isolated from other Python projects.
5. **Version Control:**
    - **Task:** Use GitHub (or a similar platform) to manage your code. Commit your code regularly and provide meaningful commit messages.

Part 4: Lab Report.

- **Content:**
    - **Introduction:** Briefly describe the problem domain, the chosen dataset, and the goals of your model.

    - **Pipeline Description:** Outline the structure of your training pipeline, including each stage and its purpose.
    - **Model Evaluation:** Present and discuss the results of your model, including metrics, performance insights, and any challenges faced.
    - **Best Practices:** Explain how you implemented the best practices encouraged in this lab, such as configuration management, logging, and code quality.
    - **Reflection:** Reflect on the effectiveness of your model and any potential improvements you would make if given more time or resources.
- **Code Submission:**
    - Submit your entire project repository, including all code, configuration files, and any saved artifacts.
    - Ensure that your GitHub repository is well-organized, with meaningful commit history and clear documentation.

Grading Criteria:

- **Code Quality and Management:** Proper use of configuration files, logging, code linters, formatters, and version control.
- **Training Pipeline Implementation:** Effective implementation of the training pipeline, including data handling, model training, and evaluation.
- **Final Results:** Quality of the model's performance metrics and presentation of results.

Resources

- Juptyter notebook - https://jupyter.org/
- Training loop with pytorch - https://pytorch.org/tutorials/beginner/blitz/cifar10_tutorial.html
- MNIST dataset - https://www.kaggle.com/datasets/hojjatk/mnist-dataset
- Rice types dataset - https://www.kaggle.com/datasets/muratkokludataset/rice-image-dataset
- Cifar dataset - https://www.cs.toronto.edu/~kriz/cifar.html

Submission Deadline:

Submit your completed project and lab report by 19.03.2026.